LAPORAN PRAKTIKUM
POSTTEST 7
ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Belva Abhinaya (2609106005)

Kelas (A1 ’26)

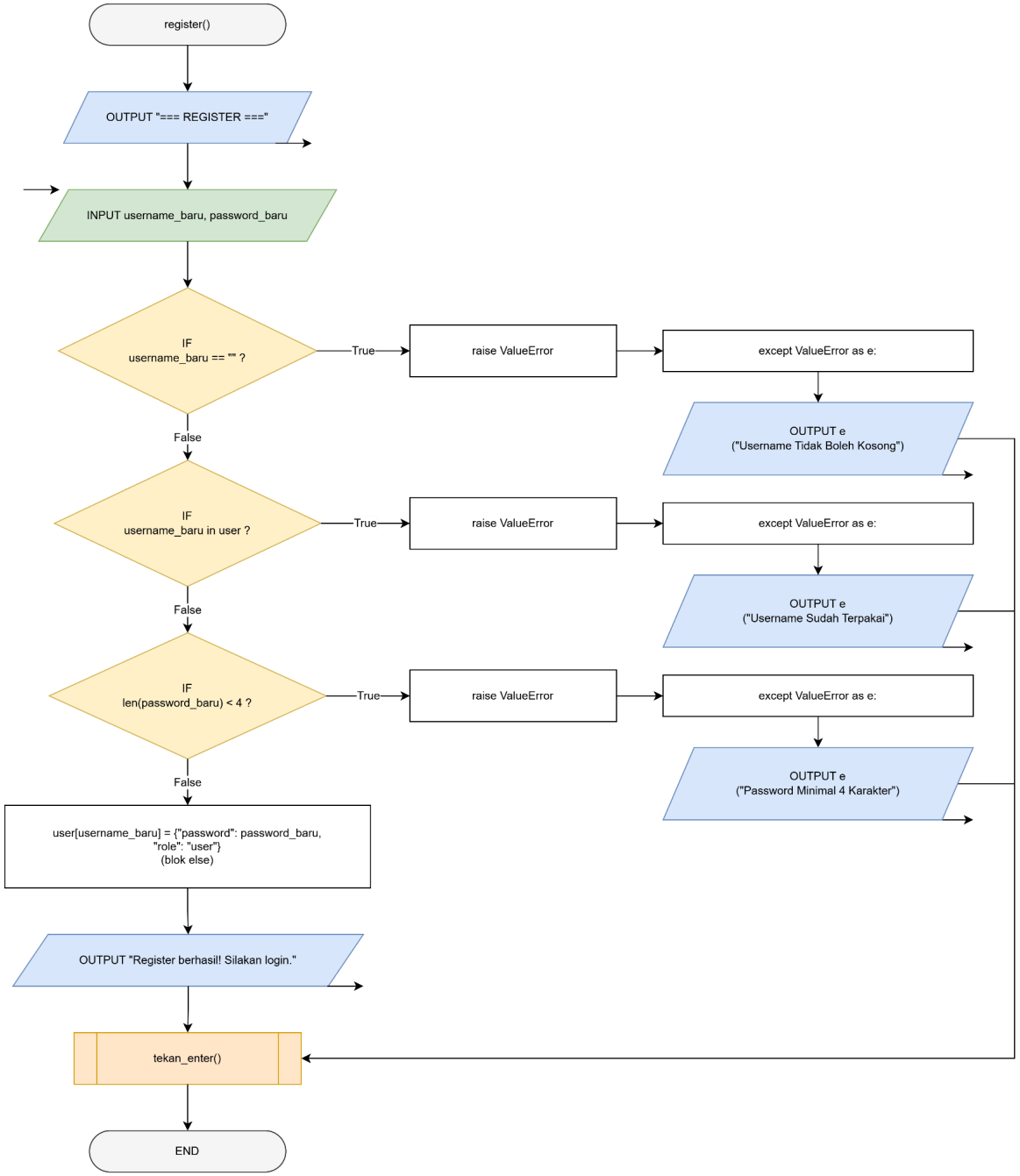
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart


register()
OUTPUT "=== REGISTER ==="
INPUT username_baru, password_baru
IF
username_baru == "" ?
True
raise ValueError
except ValueError as e:
OUTPUT e
("Username Tidak Boleh Kosong")
False
IF
username_baru in user ?
True
raise ValueError
except ValueError as e:
OUTPUT e
("Username Sudah Terpakai")
False
IF
len(password_baru) < 4 ?
True
raise ValueError
except ValueError as e:
OUTPUT e
("Password Minimal 4 Karakter")
False
user[username_baru] = {"password": password_baru,
"role": "user"}
(blok else)
OUTPUT "Register berhasil! Silakan login."
tekan_enter()
END

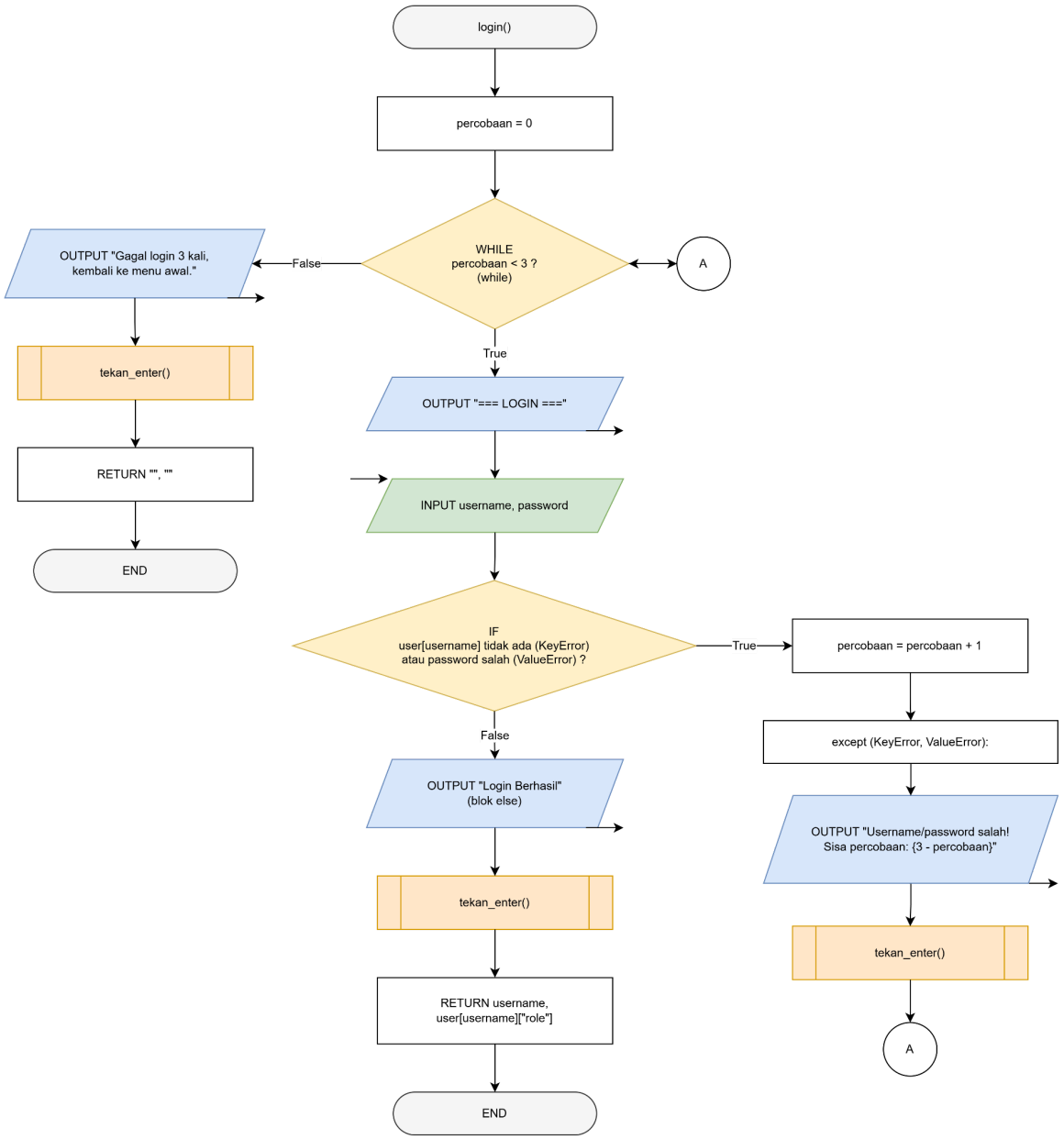
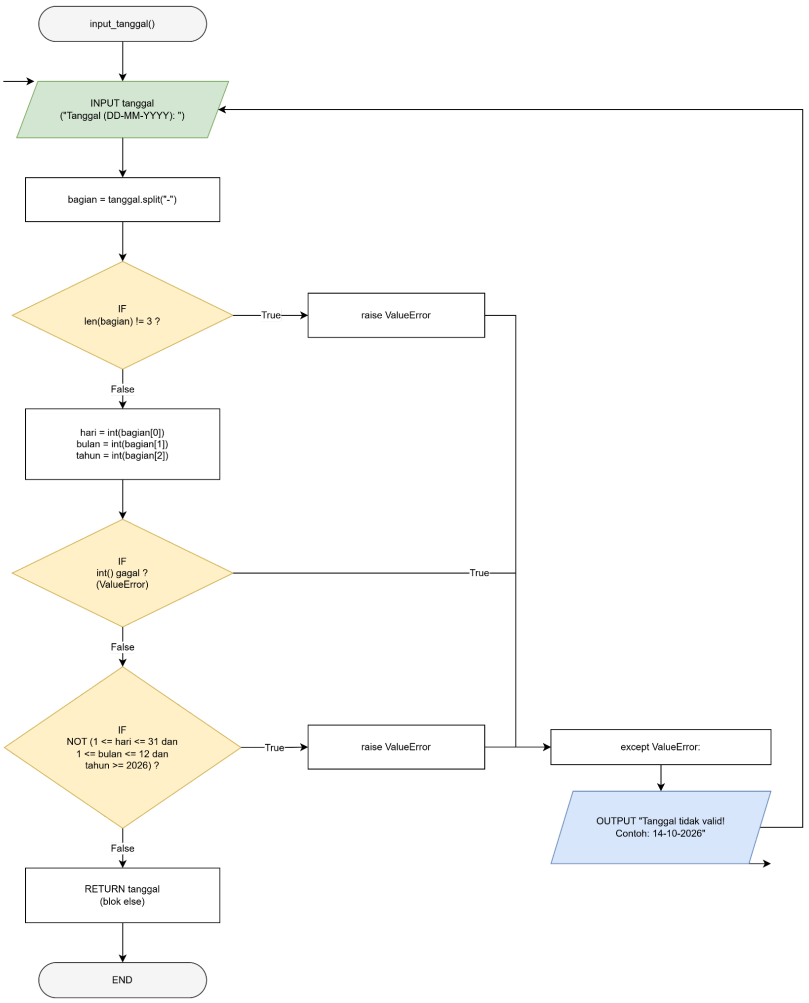
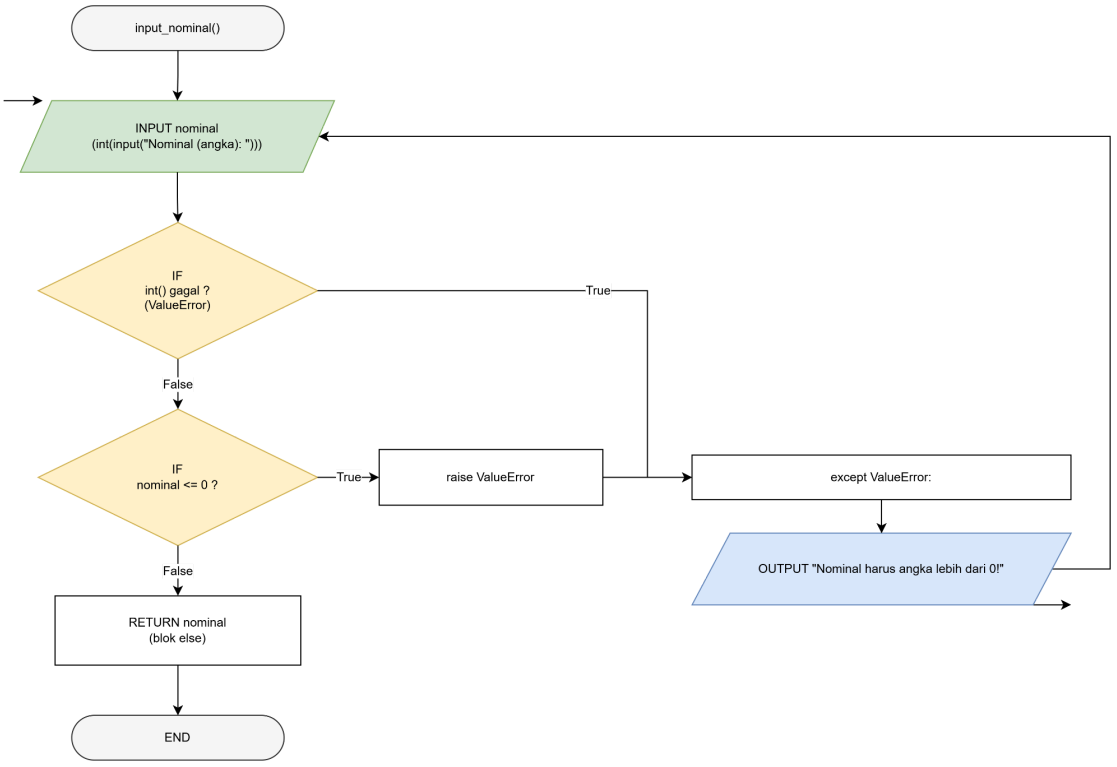


Diagram 1.1. Flowchart Prosedur register() dan Fungsi login()


input_tanggal()
INPUT tanggal
("Tanggal (DD-MM-YYYY): ")
bagian = tanggal.split("-")
IF
len(bagian) != 3 ?
True
raise ValueError
False
hari = int(bagian[0])
bulan = int(bagian[1])
tahun = int(bagian[2])
IF
int() gagal ?
(ValueError)
True
False
IF
NOT (1 <= hari <= 31 dan
1 <= bulan <= 12 dan
tahun >= 2026) ?
True
raise ValueError
except ValueError:
OUTPUT "Tanggal tidak valid!
Contoh: 14-10-2026"
False
RETURN tanggal
(blok else)
END


input_nominal()
INPUT nominal
(int(input("Nominal (angka): ")))
IF
int() gagal ?
(ValueError)
True
False
IF
nominal <= 0 ?
True
raise ValueError
except ValueError:
OUTPUT "Nominal harus angka lebih dari 0!"
False
RETURN nominal
(blok else)
END

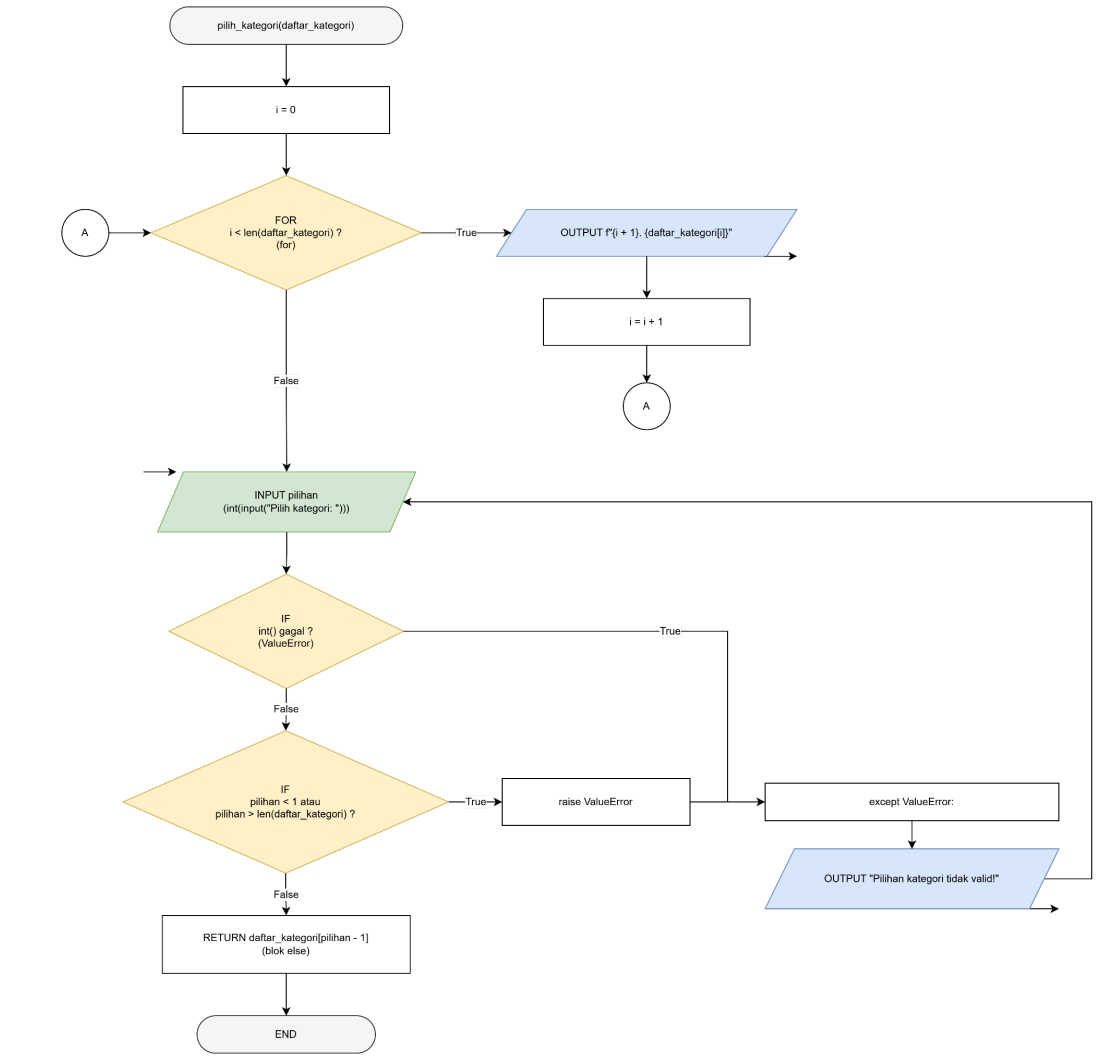
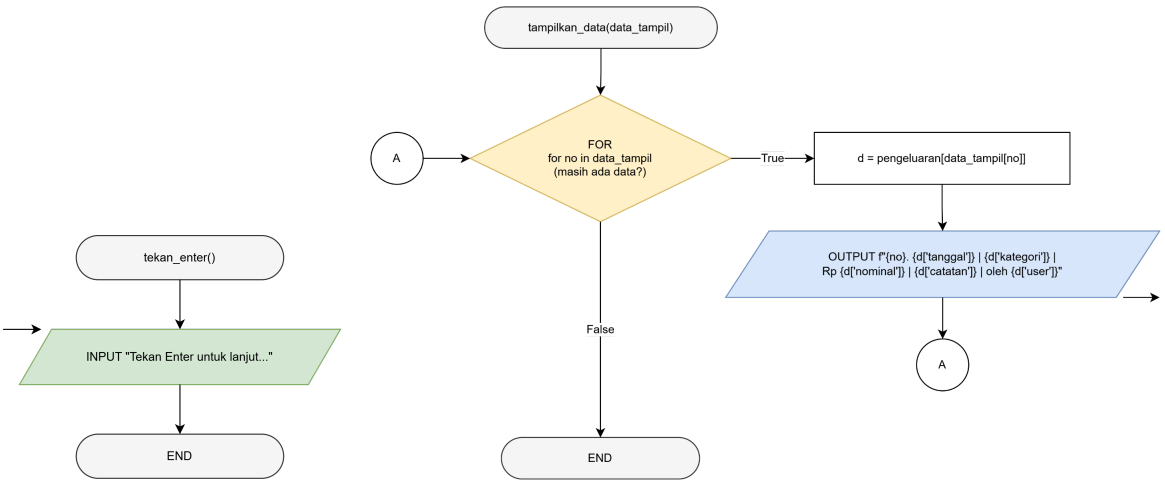
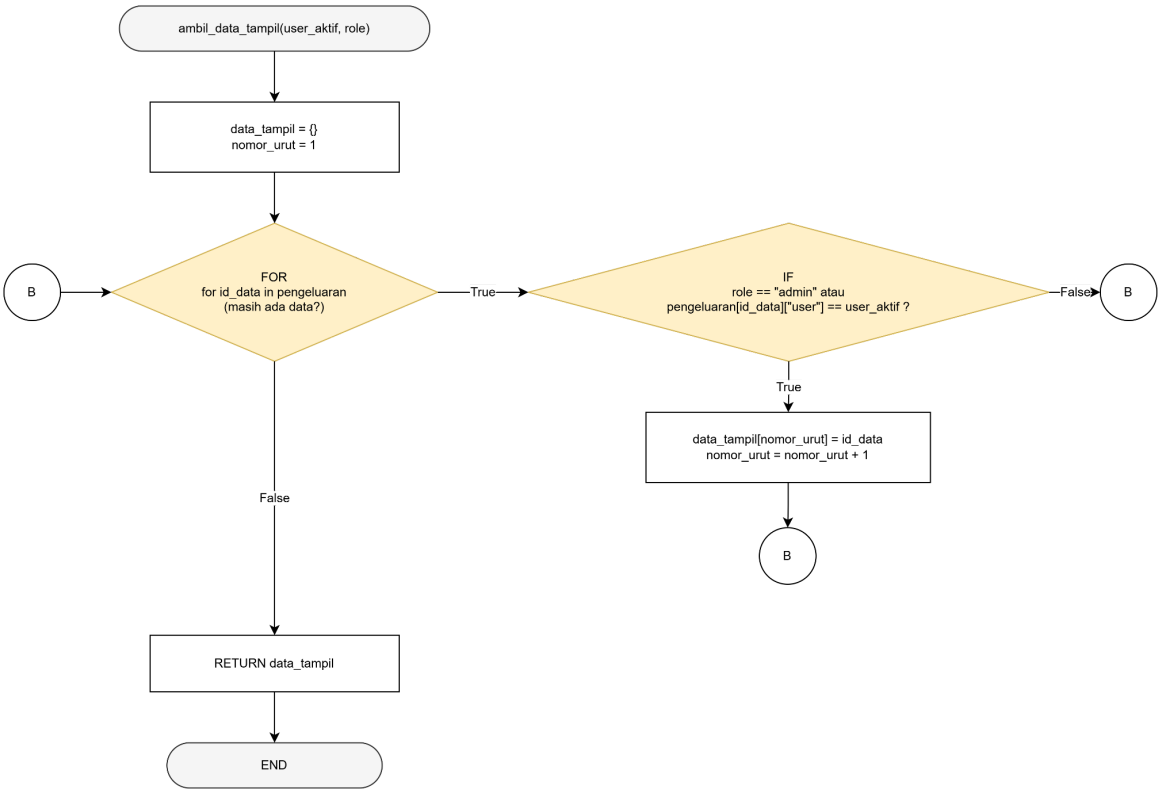
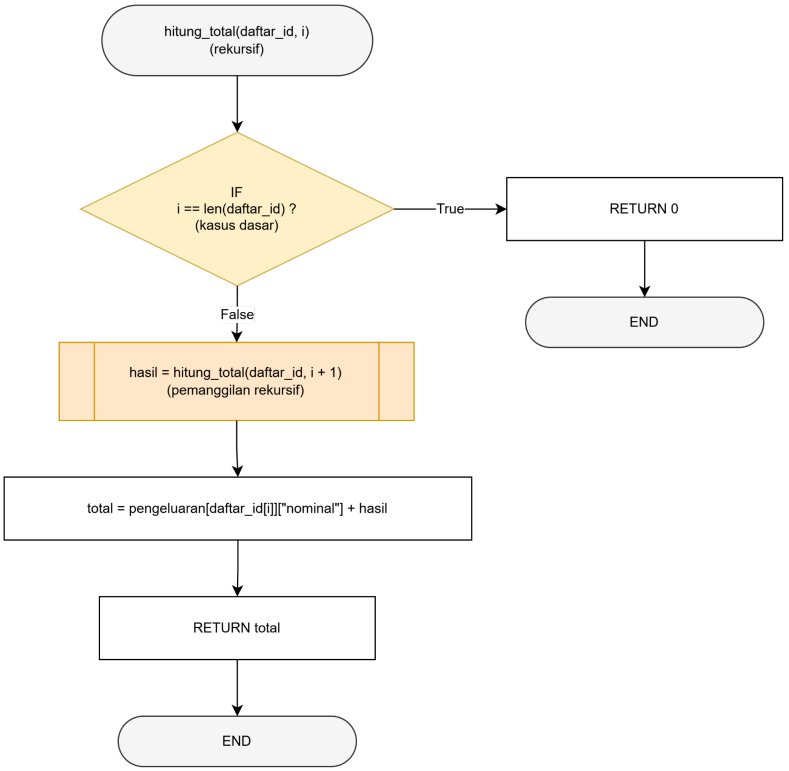


Diagram 1.2. Flowchart Fungsi input_tanggal(), input_nominal(), dan pilih_kategori()




ambil_data_tampil(user_aktif, role)
data_tampil = {}
nomor_urut = 1
B
FOR
for id_data in pengeluaran
(masih ada data?)
True
IF
role == "admin" atau
pengeluaran[id_data]["user"] == user_aktif ?
False
B
True
data_tampil[nomor_urut] = id_data
nomor_urut = nomor_urut + 1
B
False
RETURN data_tampil
END


hitung_total(daftar_id, i)
(rekursif)
IF
i == len(daftar_id) ?
(kasus dasar)
True
RETURN 0
END
False
hasil = hitung_total(daftar_id, i + 1)
(pemanggilan rekursif)
total = pengeluaran[daftar_id[i]]["nominal"] + hasil
RETURN total
END

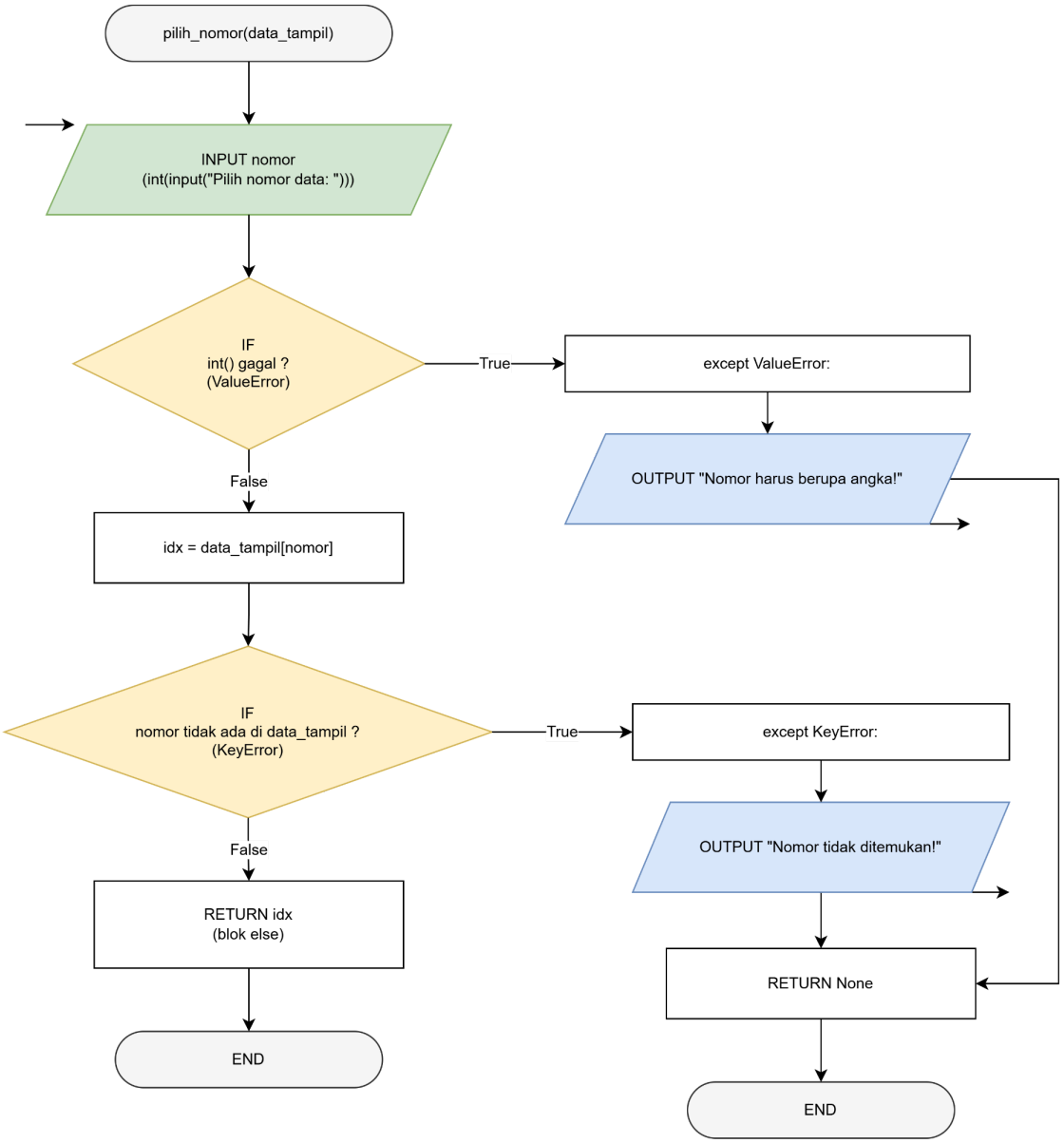
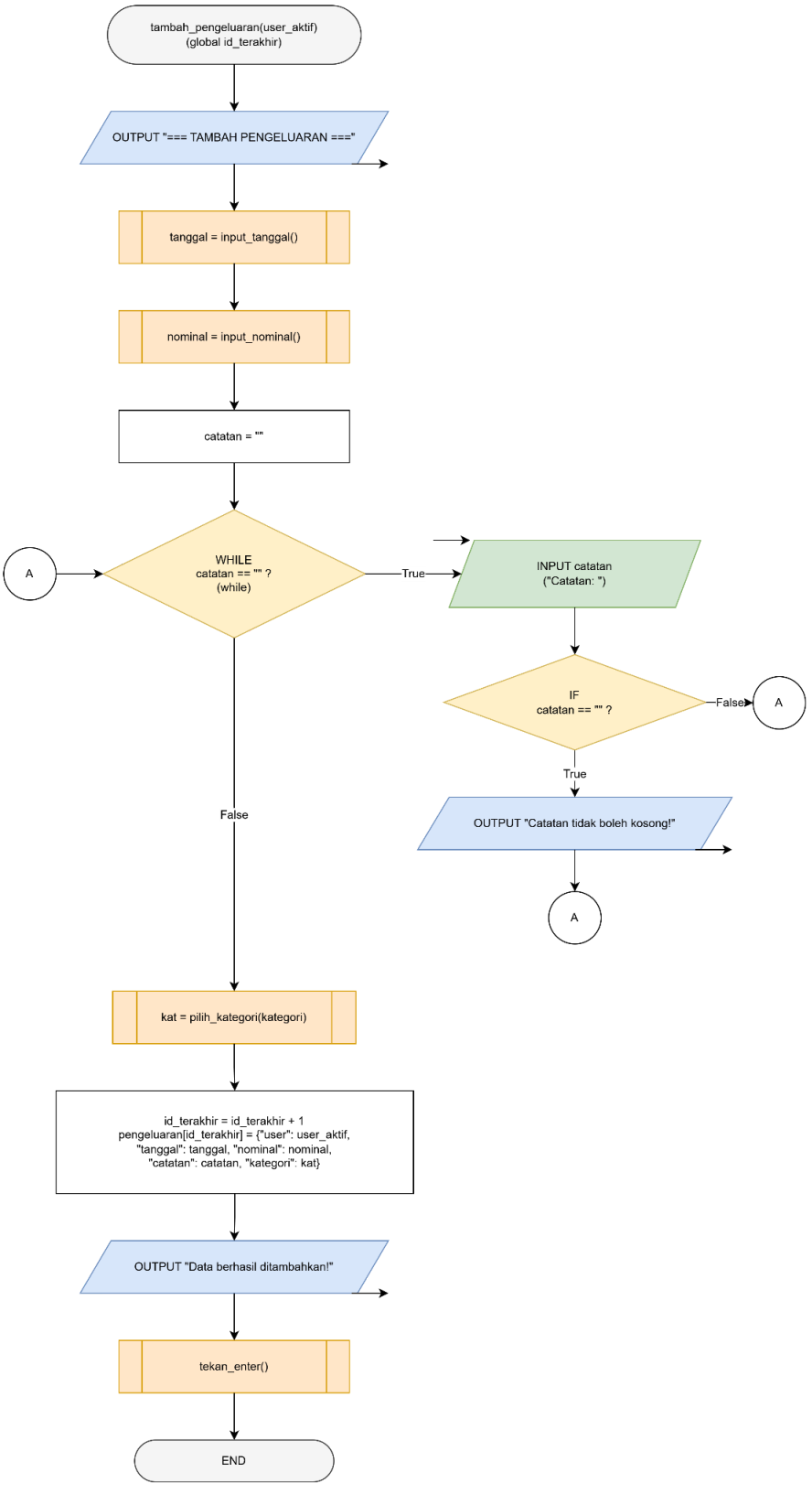
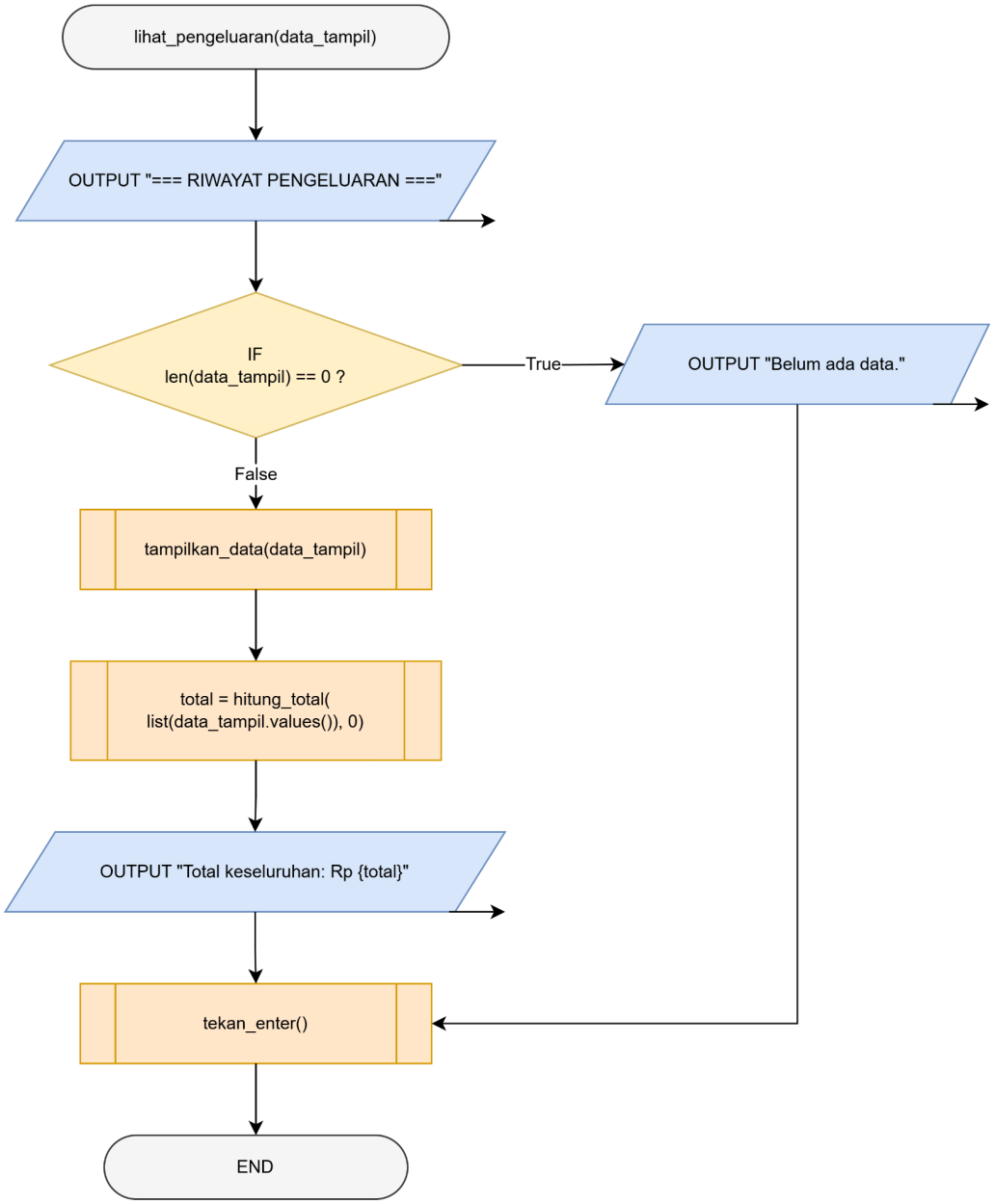


Diagram 1.3. Flowchart Prosedur tekan_enter(), tampilkan_data(), dan Fungsi ambil_data_tampil(), hitung_total(), pilih_nomor()


tambah_pengeluaran(user_aktif)
(global id_terakhir)
OUTPUT "=== TAMBAH PENGELUARAN ==="
tanggal = input_tanggal()
nominal = input_nominal()
catatan = ""
A
WHILE
catatan == "" ?
(while)
True
INPUT catatan
("Catatan: ")
IF
catatan == "" ?
False
A
True
OUTPUT "Catatan tidak boleh kosong!"
A
False
kat = pilih_kategori(kategori)
id_terakhir = id_terakhir + 1
pengeluaran[id_terakhir] = {"user": user_aktif,
"tanggal": tanggal, "nominal": nominal,
"catatan": catatan, "kategori": kat}
OUTPUT "Data berhasil ditambahkan!"
tekan_enter()
END


lihat_pengeluaran(data_tampil)
OUTPUT "=== RIWAYAT PENGELUARAN ==="
IF
len(data_tampil) == 0 ?
True
OUTPUT "Belum ada data."
False
tampilkan_data(data_tampil)
total = hitung_total(
list(data_tampil.values()), 0)
OUTPUT "Total keseluruhan: Rp {total}"
tekan_enter()
END

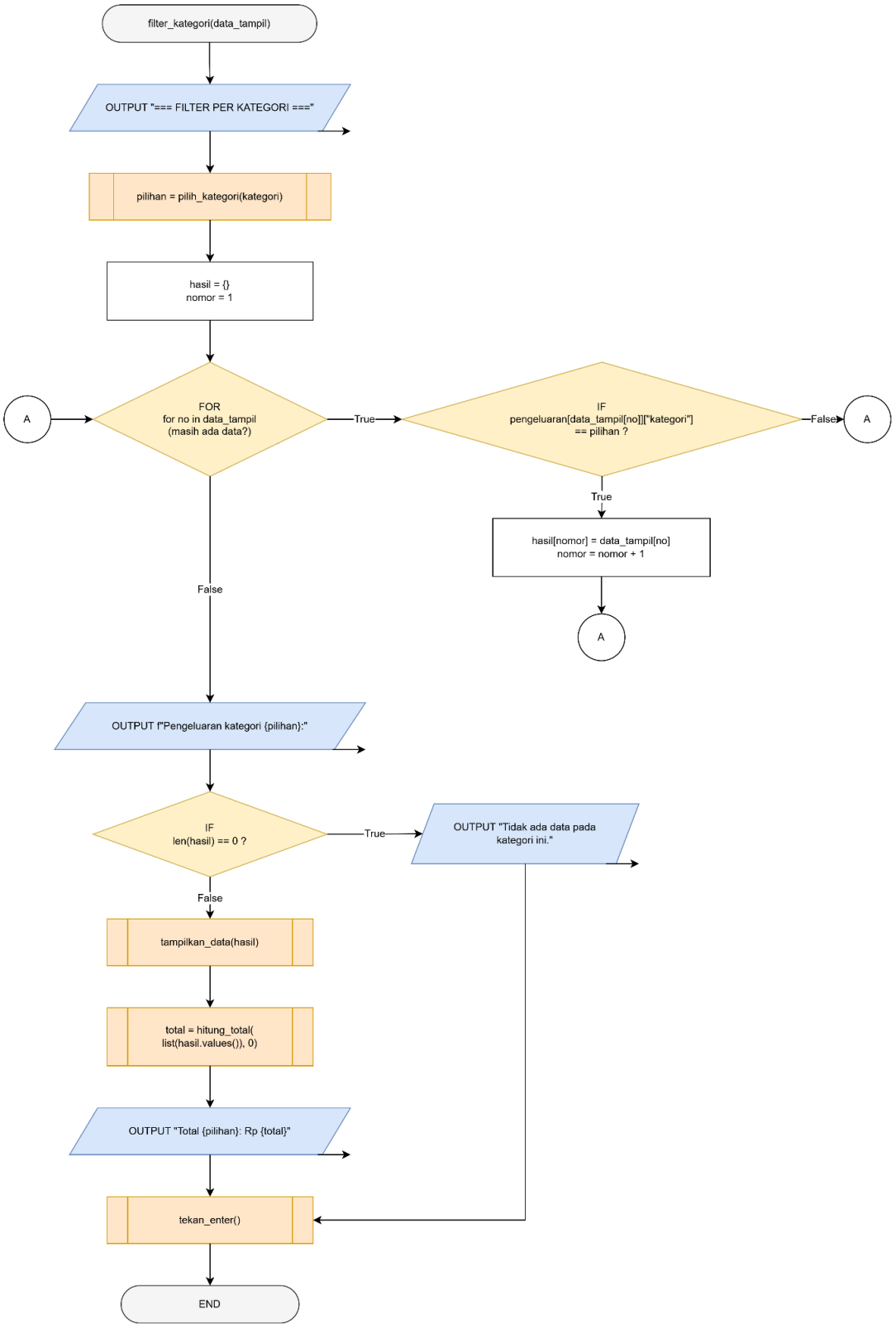
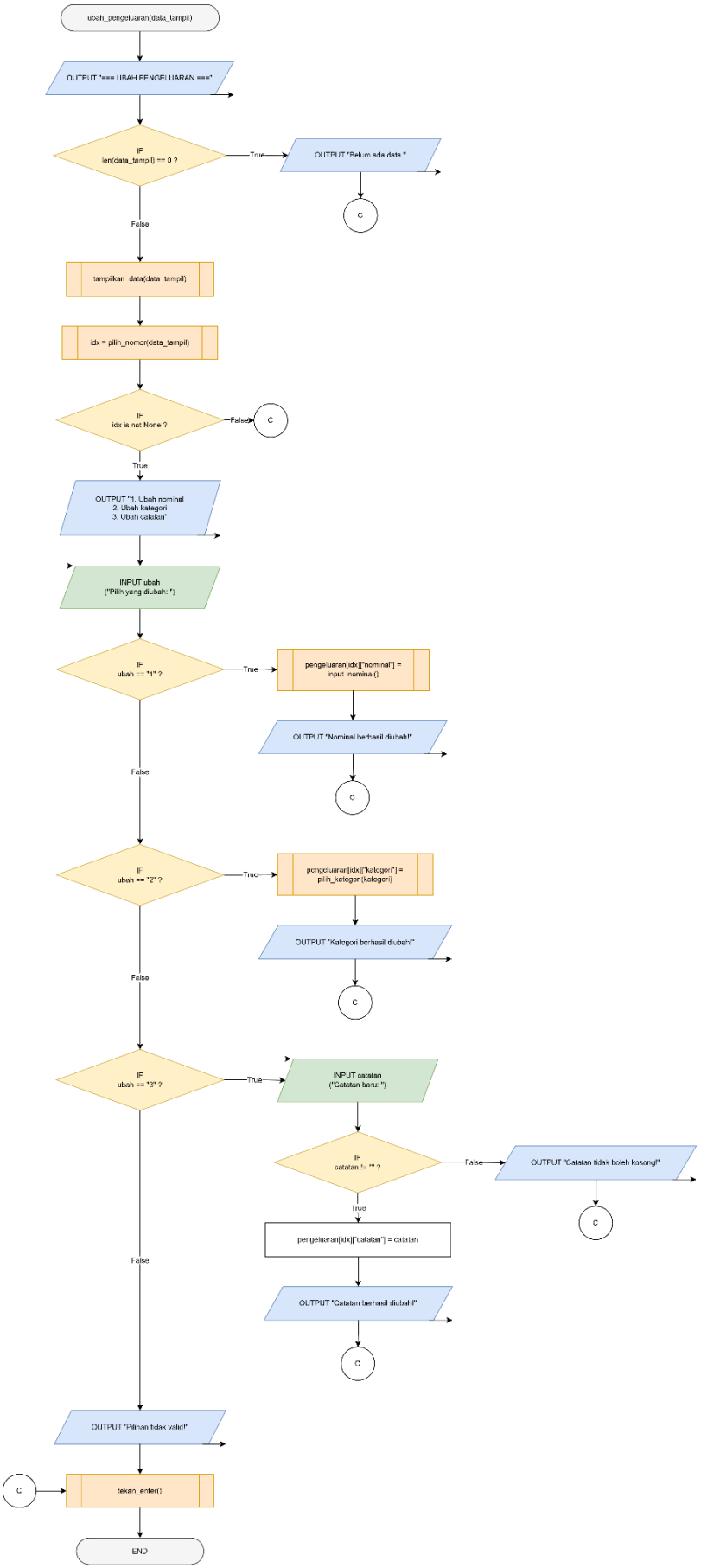


Diagram 1.4. Flowchart Prosedur tambah_pengeluaran(), lihat_pengeluaran(), dan filter_kategori()


ubah_pengeluaran(data_tampil)
OUTPUT "=== UBAH PENGELUARAN ==="
IF
len(data_tampil) == 0 ?
True
OUTPUT "Belum ada data."
C
False
tampilkan_data(data_tampil)
idx = pilih_nomor(data_tampil)
IF
idx is not None ?
False
C
True
OUTPUT "1. Ubah nominal
2. Ubah kategori
3. Ubah catatan"
INPUT ubah
("Pilih yang diubah: ")
IF
ubah == "1" ?
True
pengeluaran[idx]["nominal"] =
input_nominal()
OUTPUT "Nominal berhasil diubah!"
C
False
IF
ubah == "2" ?
True
pengeluaran[idx]["kategori"] =
pilih_kategori(kategori)
OUTPUT "Kategori berhasil diubah!"
C
False
IF
ubah == "3" ?
True
INPUT catatan
("Catatan baru: ")
IF
catatan != "" ?
False
OUTPUT "Catatan tidak boleh kosong!"
C
True
pengeluaran[idx]["catatan"] = catatan
OUTPUT "Catatan berhasil diubah!"
C
False
OUTPUT "Pilihan tidak valid!"
C
tekan_enter()
END

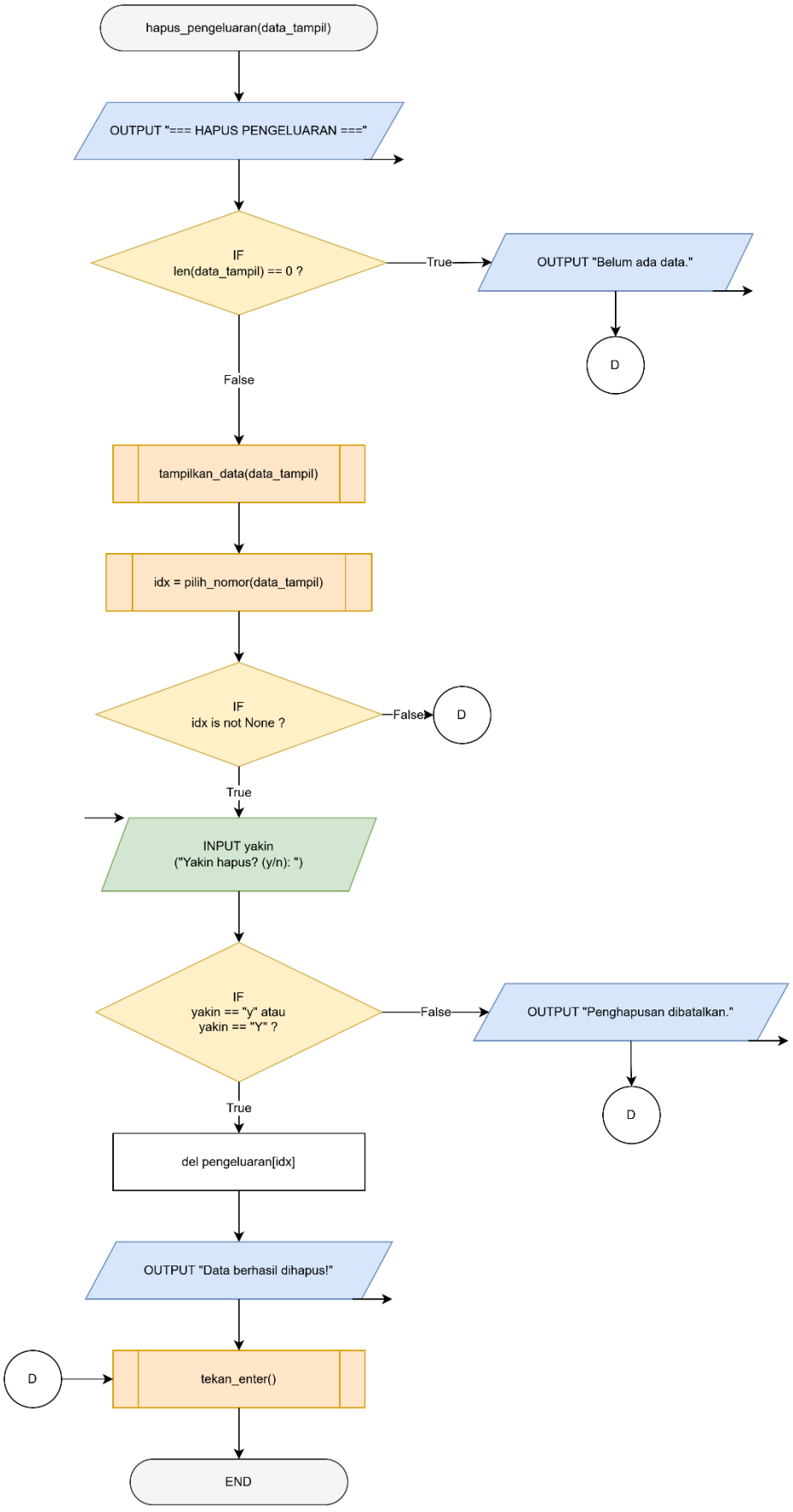


Diagram 1.5. Flowchart Prosedur ubah_pengeluaran() dan hapus_pengeluaran()

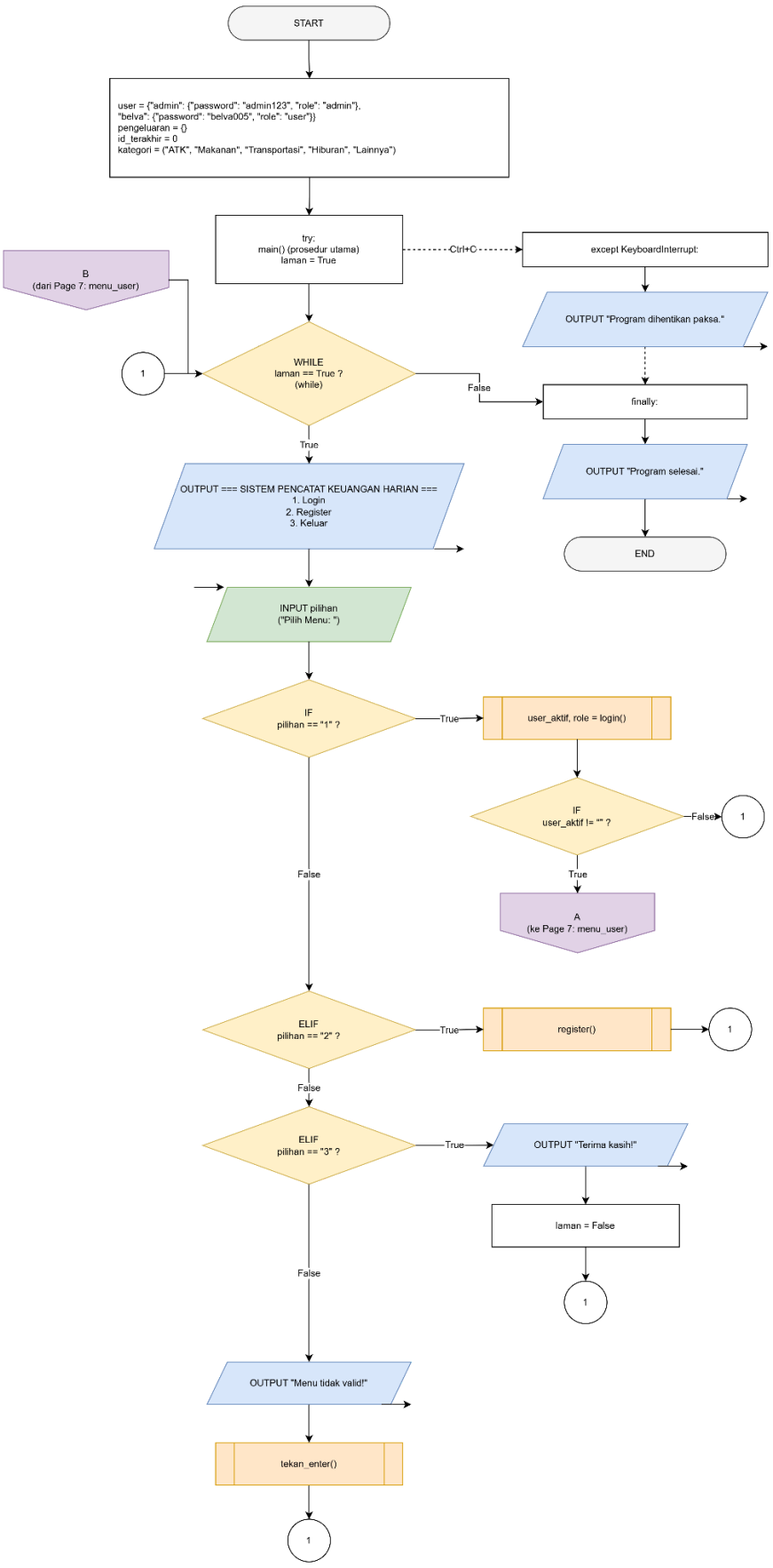


Diagram 1.6. Flowchart Program Utama main()

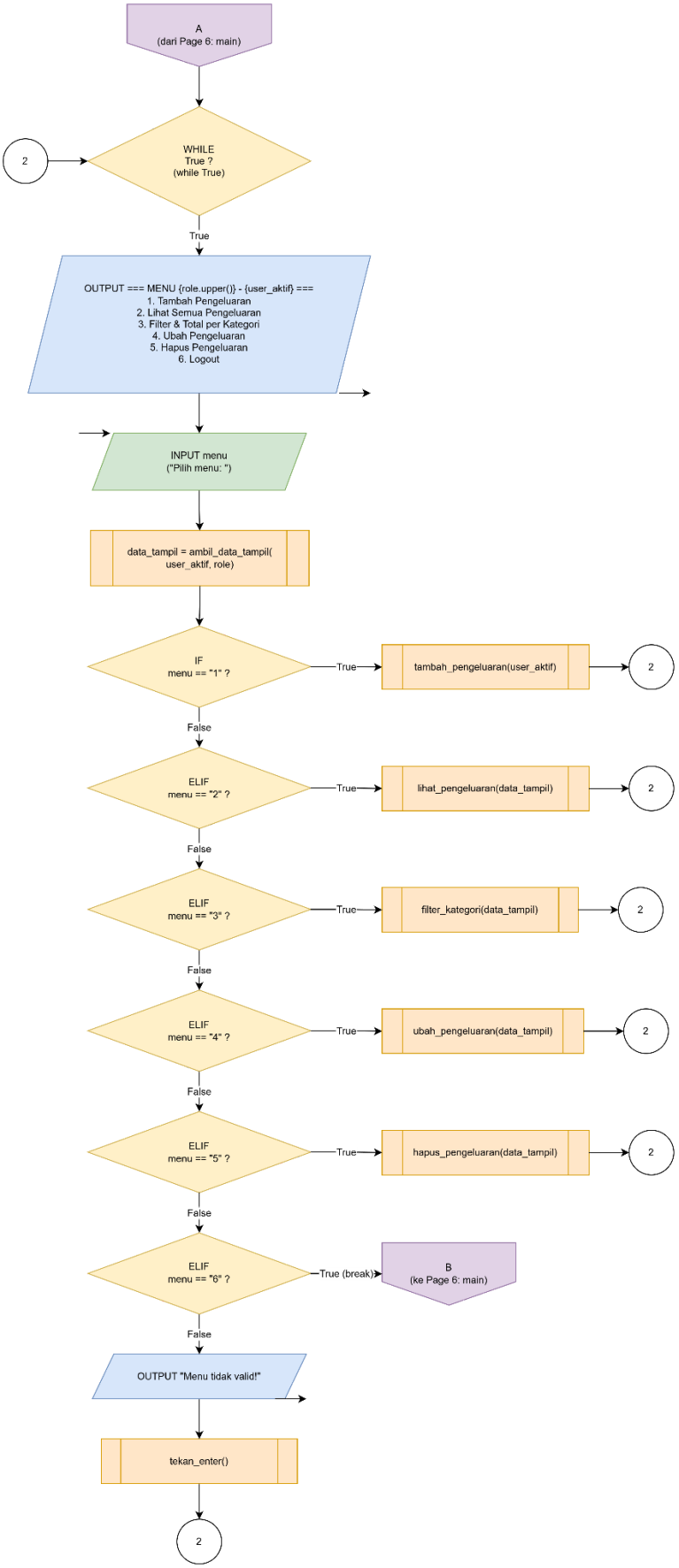


Diagram 1.7. Flowchart Prosedur menu_user(user_aktif, role)

1. Program dimulai.
2. Siapkan data awal: daftar akun (user, berisi admin dan belva beserta password dan role-nya), pengeluaran yang masih kosong, id_terakhir = 0, dan lima kategori (ATK, Makanan, Transportasi, Hiburan, Lainnya).
3. Program utama main() dijalankan di dalam try, dan laman diisi True.
4. Selama laman = True, program menampilkan menu awal (1. Login, 2. Register, 3. Keluar), lalu meminta pilihan.
5. Pilihan 1 (Login). Pengguna punya maksimal 3 kali percobaan. Tiap percobaan, program meminta username dan password.
    - Kalau username tidak ada atau password salah, percobaan bertambah 1 dan muncul pesan "Username/password salah!" beserta sisa percobaan.
    - Kalau benar, muncul "Login Berhasil", lalu program mengembalikan username dan role.
6. Kalau gagal 3 kali, muncul "Gagal login 3 kali, kembali ke menu awal." dan program kembali ke menu awal.
7. Kalau login berhasil, program masuk ke menu pengguna, yaitu menu_user().
8. Pilihan 2 (Register). Program meminta username baru dan password baru, lalu memeriksa tiga hal. Kalau salah satunya terjadi, program memunculkan error ValueError dan menampilkan pesan yang sesuai.
    - Username kosong, pesannya "Username Tidak Boleh Kosong".
    - Username sudah dipakai, pesannya "Username Sudah Terpakai".
    - Password kurang dari 4 karakter, pesannya "Password Minimal 4 Karakter".
9. Kalau semuanya aman, akun baru disimpan dengan role "user" dan muncul "Register berhasil! Silakan login."
10. Pilihan 3 (Keluar). Muncul "Terima kasih!" dan laman menjadi False, sehingga perulangan berhenti. Kalau pilihannya selain 1, 2, 3, muncul "Menu tidak valid!".
11. Di menu pengguna, program terus berulang dan menampilkan enam menu: Tambah, Lihat Semua, Filter & Total per Kategori, Ubah, Hapus, dan Logout. Pengguna lalu memilih satu menu.

12. Setiap kali ada pilihan, program menyiapkan daftar data yang boleh dilihat (data_tampil) dengan nomor urut 1, 2, 3, dan seterusnya. Admin melihat semua data, pengguna biasa hanya datanya sendiri.
13. Menu 1 (Tambah). Program meminta beberapa isian, dan kalau salah, inputnya diulang.
    - Tanggal: formatnya harus DD-MM-YYYY dengan hari 1–31, bulan 1–12, dan tahun minimal 2026.
    - Nominal: harus angka dan lebih dari 0.
    - Catatan: tidak boleh kosong.
    - Kategori: pengguna memilih dari daftar bernomor, dan nomornya harus valid.
14. Setelah itu data disimpan dan muncul “Data berhasil ditambahkan!”.
15. Menu 2 (Lihat). Kalau belum ada data, muncul “Belum ada data.”. Kalau ada, semua data ditampilkan beserta total pengeluarannya. Totalnya dihitung dengan fungsi rekursif hitung_total(), yaitu fungsi yang memanggil dirinya sendiri untuk menjumlahkan nominal satu per satu.
16. Menu 3 (Filter). Pengguna memilih satu kategori, lalu program menampilkan data kategori itu beserta totalnya. Kalau tidak ada data di kategori tersebut, muncul “Tidak ada data pada kategori ini.”.
17. Menu 4 (Ubah). Program menampilkan data, lalu pengguna memilih nomor datanya.
    - Kalau nomor bukan angka atau tidak ditemukan, muncul pesan error dan proses selesai. Input nomor tidak diulang.
    - Kalau nomor valid, pengguna memilih yang mau diubah: nominal, kategori, atau catatan. Catatan baru tidak boleh kosong.
18. Menu 5 (Hapus). Program menampilkan data, lalu pengguna memilih nomor datanya dan ditanya “Yakin hapus? (y/n)”. Kalau jawabannya y atau Y, data dihapus. Kalau bukan, muncul “Penghapusan dibatalkan.”.
19. Menu 6 (Logout). Program keluar dari menu pengguna dan kembali ke menu awal. Kalau menu selain 1–6, muncul “Menu tidak valid!”.

20. Kalau program dihentikan paksa dengan Ctrl+C, muncul “Program dihentikan paksa.”. Setelah itu, baik berhenti normal maupun paksa, finally selalu menampilkan “Program selesai.”.
21. Selesai.

2. Deskripsi Singkat Program

Program ini adalah Sistem Pencatat Keuangan Harian yang dibuat untuk membantu pengguna mencatat dan mengelola pengeluaran sehari-hari. Program menyediakan fitur register dan login supaya setiap pengguna memiliki akun sendiri. Terdapat dua jenis pengguna, yaitu admin dan pengguna biasa. Admin dapat melihat serta mengelola seluruh data pengeluaran, sedangkan pengguna biasa hanya dapat melihat dan mengelola data miliknya sendiri. Percobaan login dibatasi maksimal tiga kali.

Pada Posttest 7, program dari Posttest 6 diubah agar lebih terstruktur. Setiap fitur dipisah menjadi fungsi dan prosedur sehingga kode lebih pendek, mudah dibaca, dan mudah dirawat. Fungsi digunakan untuk pekerjaan yang menghasilkan nilai (seperti input tanggal, input nominal, memilih kategori, dan menghitung total), sedangkan prosedur digunakan untuk pekerjaan yang hanya menjalankan blok kode tanpa mengembalikan nilai (seperti menampilkan data, tambah, ubah, dan hapus). Perhitungan total pengeluaran memakai fungsi rekursif.

Program juga menerapkan error handling menggunakan try-except, else, finally, dan raise. Dengan cara ini program tidak berhenti mendadak (crash) ketika pengguna memasukkan input yang salah, misalnya huruf pada kolom nominal atau format tanggal yang keliru. Konsep CRUD (Create, Read, Update, Delete) tetap diterapkan dengan dictionary dan nested dictionary.

3. Source Code

A. Fungsi Tanpa Parameter

Fungsi tanpa parameter adalah fungsi yang tidak memerlukan data dari luar saat dipanggil dan mengembalikan nilai dengan return. Pada program ini terdapat fungsi input_tanggal() yang mengembalikan tanggal valid dan input_nominal() yang mengembalikan nominal valid. Keduanya mengulang input sampai benar.

```
def input_tanggal():
    while True:
        try:
            tanggal = input("Tanggal (DD-MM-YYYY): ")
            bagian = tanggal.split("-")
            if len(bagian) != 3:
                raise ValueError
            hari = int(bagian[0])
            bulan = int(bagian[1])
            tahun = int(bagian[2])
            if not (1 <= hari <= 31 and 1 <= bulan <= 12 and tahun >= 2026):
                raise ValueError
        except ValueError:
            print("Tanggal tidak valid! Contoh: 14-10-2026")
        else:
            return tanggal
```

```
def input_nominal():
    while True:
        try:
            nominal = int(input("Nominal (angka): "))
            if nominal <= 0:
                raise ValueError

        except ValueError:
            print("Nominal harus angka lebih dari 0!")
        else:
            return nominal
```

B. Fungsi Dengan Parameter

Fungsi dengan parameter menerima nilai (argumen) dari luar untuk diproses. Fungsi pilih_kategori() menerima daftar_kategori lalu mengembalikan kategori yang dipilih. Fungsi ambil_data_tampil() menerima user_aktif dan role lalu mengembalikan dictionary data yang boleh dilihat pengguna tersebut.

```
def pilih_kategori(daftar_kategori):
    for i in range(len(daftar_kategori)):
        print(f"{i + 1}. {daftar_kategori[i]}")
    while True:
        try:
            pilihan = int(input("Pilih kategori: "))
            if pilihan < 1 or pilihan > len(daftar_kategori):
                raise ValueError
        except ValueError:
            print("Pilihan kategori tidak valid!")
        else:
            return daftar_kategori[pilihan - 1]
```

```
def ambil_data_tampil(user_aktif, role):
    data_tampil = {}
    nomor_urut = 1
    for id_data in pengeluaran:
        if role == "admin" or pengeluaran[id_data]["user"] == user_aktif:
            data_tampil[nomor_urut] = id_data
            nomor_urut += 1
    return data_tampil
```

C. Fungsi Rekursif

Fungsi Rekursif adalah fungsi yang memecahkan masalah dengan cara memanggil dirinya sendiri hingga kondisi tertentu, tetapi dengan argumen yang berbeda, untuk menyelesaikan sub-masalah yang lebih kecil.

```
def ambil_data_tampil(user_aktif, role):
    data_tampil = {}
    nomor_urut = 1
    for id_data in pengeluaran:
        if role == "admin" or pengeluaran[id_data]["user"] == user_aktif:
            data_tampil[nomor_urut] = id_data
            nomor_urut += 1
```

```
    return data_tampil
```

D. Prosedur

Prosedur adalah fungsi yang menjalankan blok kode tanpa mengembalikan nilai (tidak memakai return). Contohnya prosedur tekan_enter() dan tampilkan_data() yang hanya mencetak tampilan ke layar.

```
def tekan_enter():
    input("Tekan Enter untuk lanjut...")
```

```
def tampilkan_data(data_tampil):
    for no in data_tampil:
        d = pengeluaran[data_tampil[no]]
        print(f"{no}. {d['tanggal']} | {d['kategori']} | Rp {d['nominal']} |
{d['catatan']} | oleh {d['user']}")
```

E. Fitur Input Data

Fitur input digunakan untuk menerima data dari pengguna, seperti username, password, tanggal, nominal, catatan, kategori, dan pilihan menu.

```
    username_baru = input("Masukkan Username Baru: ")
    password_baru = input("Masukkan Password Baru: ")
    catatan = input("Catatan: ")
    menu = input("Pilih menu: ")
```

F. Fitur Percabangan (IF/ELIF/ELSE)

Percabangan digunakan untuk menentukan tindakan berdasarkan kondisi, yaitu memeriksa pilihan menu dan memanggil fungsi atau prosedur yang sesuai.

```
  if menu == "1":
            tambah_pengeluaran(user_aktif)
        elif menu == "2":
            lihat_pengeluaran(data_tampil)
```

```
        elif menu == "3":
            filter_kategori(data_tampil)
        elif menu == "4":
            ubah_pengeluaran(data_tampil)
        elif menu == "5":
            hapus_pengeluaran(data_tampil)
        elif menu == "6":
            break
        else:
            print("Menu tidak valid!")
            tekan_enter()
```

G. Fitur Perulangan (FOR dan WHILE)

Perulangan while digunakan untuk menampilkan menu berulang dan mengulang input sampai valid. Perulangan for digunakan untuk membaca data pengeluaran dan menampilkan kategori.

```
def main():
    laman = True
    while laman:
        os.system('cls')
        print("=== SISTEM PENCATAT KEUANGAN HARIAN ===")
        print("1. Login")
        print("2. Register")
        print("3. Keluar")
        pilihan = input("Pilih Menu: ")

        if pilihan == "1":
            user_aktif, role = login()
            if user_aktif != "":
                menu_user(user_aktif, role)
        elif pilihan == "2":
            register()
        elif pilihan == "3":
            print("Terima kasih!")
            laman = False
        else:
            print("Menu tidak valid!")
            tekan_enter()
```

H. Fitur Dictionary, Tuple, dan Nested Dictionary

Dictionary menyimpan data berpasangan key dan value. Data akun memakai username sebagai key dengan value berisi password dan role. Data pengeluaran adalah nested dictionary dengan id transaksi sebagai key dan value berisi pemilik, tanggal, nominal, catatan, dan kategori. Tuple dipakai untuk daftar kategori yang tidak berubah.

```
ser = {
    "admin": {"password": "admin123", "role": "admin"},
    "belva": {"password": "belva005", "role": "user"}
}
pengeluaran = {}
id_terakhir = 0
kategori = ("ATK", "Makanan", "Transportasi", "Hiburan", "Lainnya")
```

I. Fitur CRUD

CRUD adalah bagian utama program. Create menambah pasangan key dan value baru ke dictionary pengeluaran. Read menampilkan riwayat dan total. Update mengubah value pada key tertentu. Delete menghapus data dengan del.

1). Create

```
 id_terakhir += 1
    pengeluaran[id_terakhir] = {
        "user": user_aktif,
        "tanggal": tanggal,
        "nominal": nominal,
        "catatan": catatan,
        "kategori": kat
    }
```

2). Read

```
def lihat_pengeluaran(data_tampil):
    os.system('cls')
    print("=== RIWAYAT PENGELUARAN ===")
    if len(data_tampil) == 0:
```

```
        print("Belum ada data.")
    else:
        tampilkan_data(data_tampil)
        total = hitung_total(list(data_tampil.values()), 0)
        print(f"Total keseluruhan: Rp {total}")
    tekan_enter()
```

3). Update

```
            if ubah == "1":
                pengeluaran[idx]["nominal"] = input_nominal()
                print("Nominal berhasil diubah!")
            elif ubah == "2":
                pengeluaran[idx]["kategori"] = pilih_kategori(kategori)
                print("Kategori berhasil diubah!")
            elif ubah == "3":
                catatan = input("Catatan baru: ")
                if catatan != "":
                    pengeluaran[idx]["catatan"] = catatan
                    print("Catatan berhasil diubah!")
```

4). Delete

```
 if yakin == "y" or yakin == "Y":
                del pengeluaran[idx]
```

J. Fitur Error Handling

Error handling mencegah program berhenti mendadak akibat input yang salah. Blok try berisi kode yang mungkin menimbulkan error, blok except menangani error tertentu (ValueError, KeyError), blok else dijalankan jika tidak ada error, dan blok finally selalu dijalankan. Kata kunci raise dipakai untuk membuat error secara manual saat validasi gagal.

```
def pilih_nomor(data_tampil):
    try:
        nomor = int(input("Pilih nomor data: "))
        idx = data_tampil[nomor]
    except ValueError:
        print("Nomor harus berupa angka!")
        return None
    except KeyError:
        print("Nomor tidak ditemukan!")
```

```
        return None
    else:
        return idx
```

```
def register():
    os.system('cls')
    print("=== REGISTER ===")
    username_baru = input("Masukkan Username Baru: ")
    password_baru = input("Masukkan Password Baru: ")
    try:
        if username_baru == "":
            raise ValueError("Username Tidak Boleh Kosong")
        if username_baru in user:
            raise ValueError("Username Sudah Terpakai")
        if len(password_baru) < 4:
            raise ValueError("Password Minimal 4 Karakter")
    except ValueError as e:
        print(e)
    else:
        user[username_baru] = {"password": password_baru, "role": "user"}
        print("Register berhasil! Silakan login.")
    tekan_enter()
```

K. Fitur Clear Terminal dengan Library OS

Library os digunakan untuk membersihkan terminal setiap berpindah menu. Perintah os.system('cls') digunakan pada terminal Windows.

```
import os
os.system('cls')
```

L. Fitur Output (Print dan F-String)

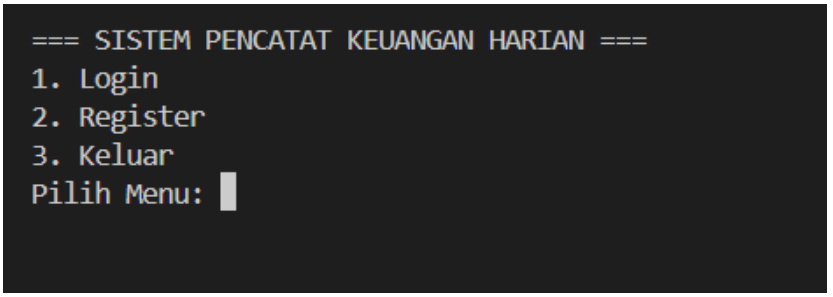
Fitur output menampilkan menu, riwayat, total, pesan validasi, dan pesan keberhasilan. F-string menampilkan nilai variabel langsung di dalam teks.

```
print(f"=== MENU {role.upper()} - {user_aktif} ===")
print("1. Tambah Pengeluaran")
print("2. Lihat Semua Pengeluaran")
```

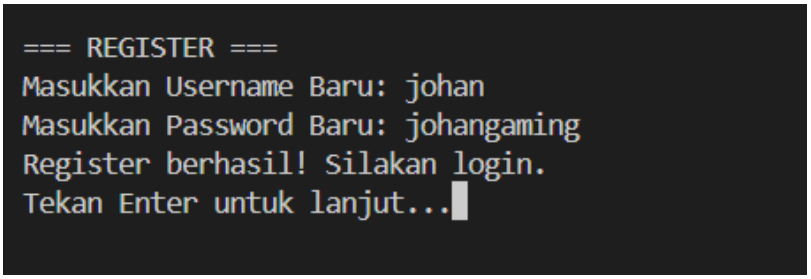
```
print("3. Filter & Total per Kategori")
print("4. Ubah Pengeluaran")
print("5. Hapus Pengeluaran")
print("6. Logout")
```

```
print(f"Total keseluruhan: Rp {total}")
print(f"Username/password salah! Sisa percobaan: {3 - percobaan}")
```

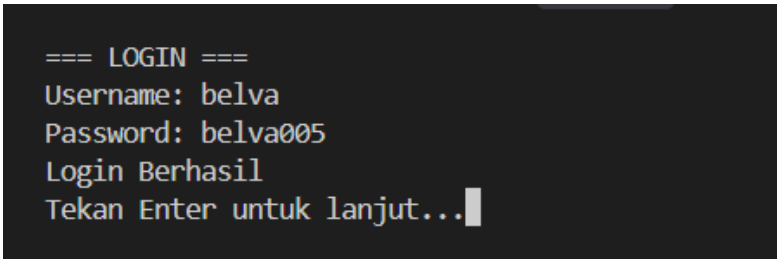
4. Hasil Output



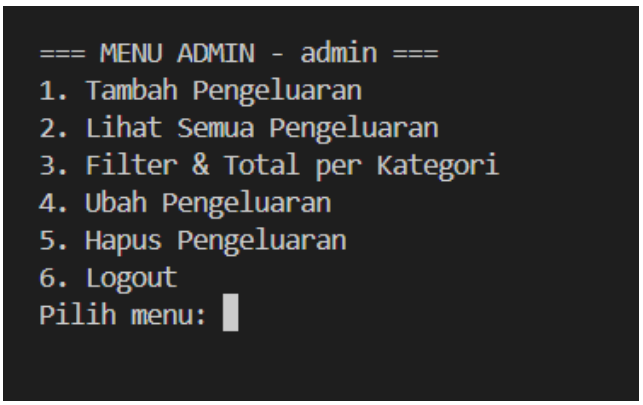
Gambar 4.1 Tampilan Menu Utama



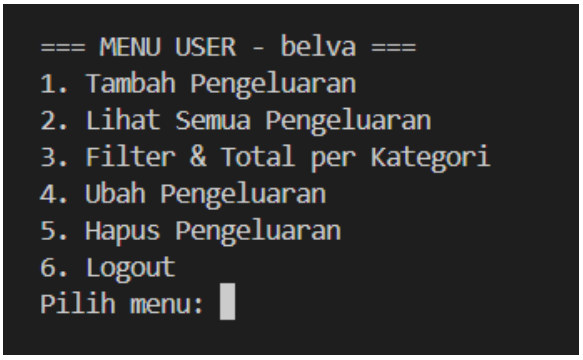
Gambar 4.2 Tampilan Register



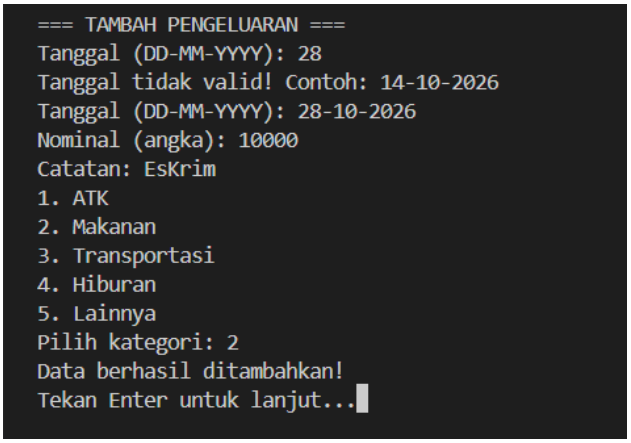
Gambar 4.3 Tampilan Jika Login Berhasil



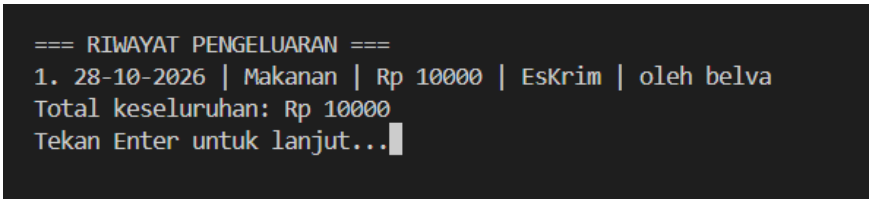
Gambar 4.4 Tampilan Menu Admin



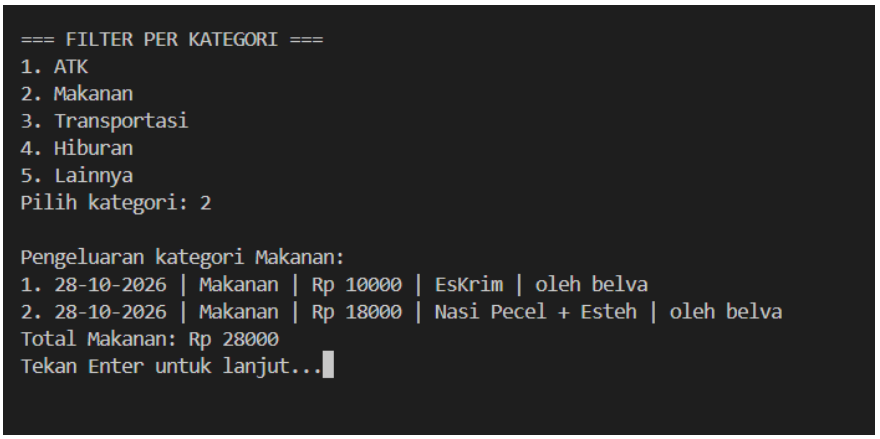
Gambar 4.5 Tampilan Menu Pengguna Biasa



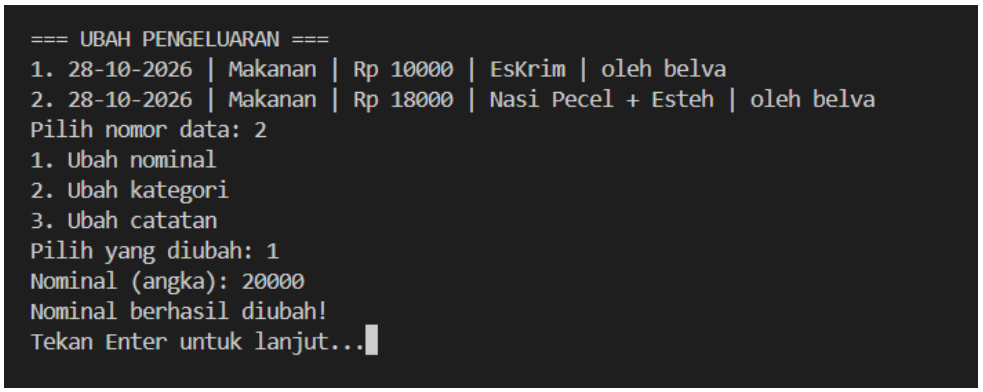
Gambar 4.6 Tampilan Menu Tambah Pengeluaran



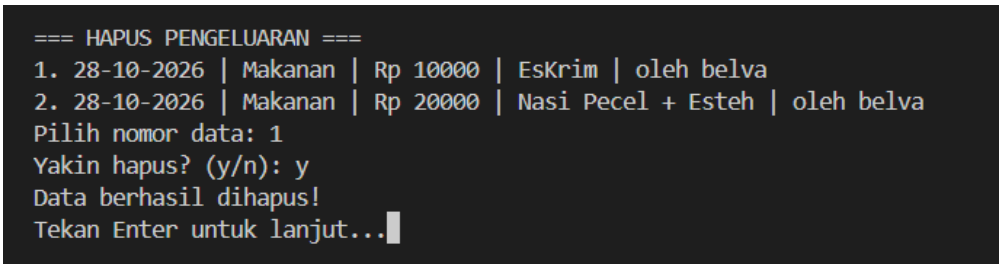
Gambar 4.7 Tampilan Menu Riwayat Pengeluaran



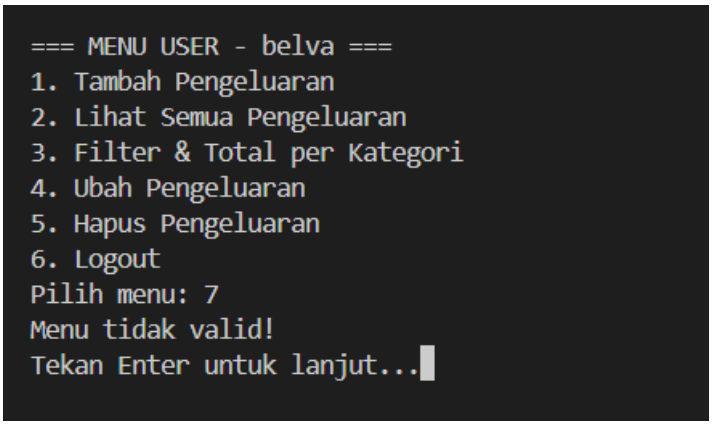
Gambar 4.8 Tampilan Menu Filter dan Total per Kategori



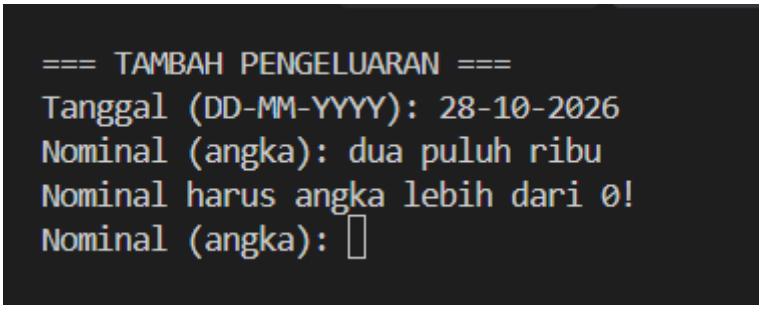
Gambar 4.9 Tampilan Menu Ubah Pengeluaran



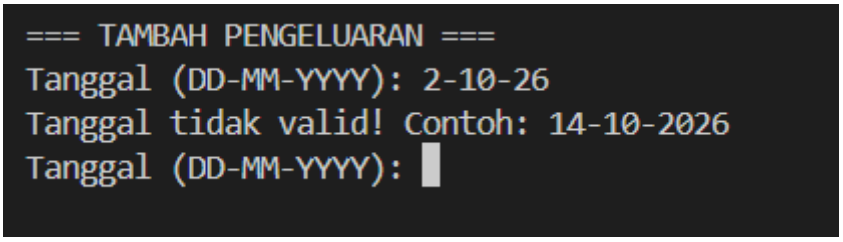
Gambar 4.10 Tampilan Menu Hapus Pengeluaran



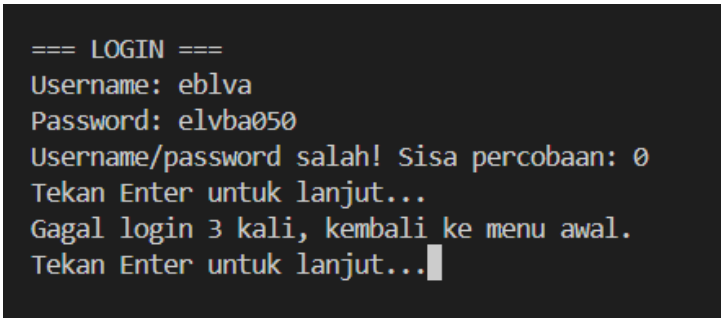
Gambar 4.11 Output Validasi Input Tidak Valid (menu salah)



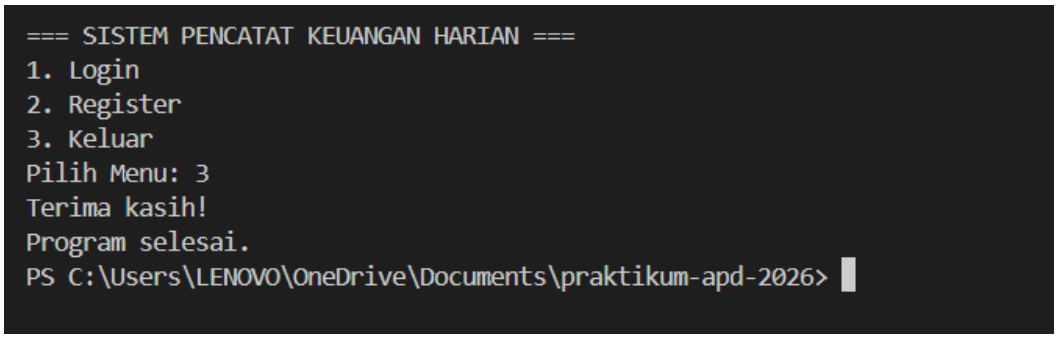
Gambar 4.12 Output Error Handling Nominal Bukan Angka



Gambar 4.13 Output Error Handling Format Tanggal Salah



Gambar 4.14 Output Gagal Login 3 Kali



Gambar 4.15 Output Keluar dari Program

5.1 Git Add

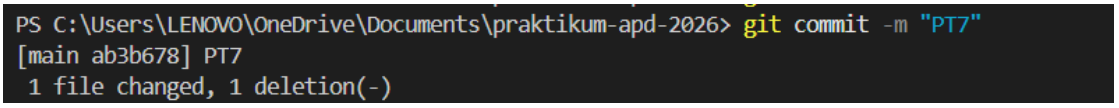
Perintah git add digunakan untuk menambahkan file baru atau perubahan file ke dalam staging area Git sebelum disimpan melalui commit. Perintah ini dapat digunakan untuk memilih file tertentu atau menambahkan semua perubahan.



Gambar 5.1 Git Add

5.2 Git Commit

Perintah git commit digunakan untuk menyimpan perubahan yang sudah dimasukkan ke staging area sebagai sebuah riwayat perubahan di repositori lokal. Setiap commit dapat diberi pesan untuk menjelaskan perubahan yang dilakukan.



Gambar 5.2 Git Commit

5.3 Git Push

Perintah git push digunakan untuk mengirimkan commit dari repositori lokal ke repositori jarak jauh, misalnya GitHub. Dengan perintah ini, perubahan program dapat diunggah dan disimpan di repositori online.

```
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-apd-2026> git push --set-upstream origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 480 bytes | 480.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/BelvaAbhinaya005/praktikum-apd-2026.git
   4b6b308..ab3b678  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-apd-2026>
```

Gambar 5.3 Git Push